

Inductive Bias & Distillation.

A CNN gets locality and translation equivariance for free; a ViT must learn them from data, which is why it is data-hungry. DeiT closes the gap with a distillation token and a two-head loss. This topic derives the permutation argument and computes a full distillation loss by hand — nothing summarised or deferred.

THE EQUATION

$$\mathcal{L}_{\text{hard}} = \frac{1}{2} \text{CE}(\psi(z_{\text{cls}}), y) + \frac{1}{2} \text{CE}(\psi(z_{\text{dist}}), y_t), \quad y_t = \arg \max_c f_{\text{teacher}}(x)_c$$

Worked example. A 3-class problem with student class-head and distillation-head logits and a CNN teacher's argmax label; both cross-entropy terms and the averaged loss computed in full.

Topic 2 of 2 in this module · companion to the course page · v1.0

Contents

1	The claim: ViT lacks a CNN's inductive bias	2
2	Why parameter sharing matters, and why attention is permutation-equivariant	3
3	DeiT: distillation through a dedicated token	3
4	The derivation: a 3-class distillation loss, computed in full	4
5	Properties and consequences	5
6	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 The claim: ViT lacks a CNN's inductive bias

A convolutional network bakes two assumptions into its architecture before it sees any data: *locality* (a unit looks only at a small neighbourhood) and *translation equivariance* (the same kernel weights slide across all spatial positions, so a shifted input produces a shifted output). These are *inductive biases*: hard constraints that shrink the hypothesis space toward solutions that tend to generalise on images.

A ViT has neither. Self-attention lets any patch attend to any other from layer one — it is global, and its weights are not tied to spatial position. The model therefore has to *learn* spatial structure from scratch. With enough data (e.g. JFT-300M, ~300M images) ViT learns better features than a CNN and wins; on ImageNet-1k alone (~1.3M images) it underfits the missing prior and *underperforms* comparable CNNs.

$$\begin{array}{l} \text{conv: weights shared across positions} \Rightarrow \text{equivariant} + \text{cheap} \\ \text{attn: global, position-free} \Rightarrow \text{flexible} + \text{data-hungry} \end{array} \quad (1)$$

Inductive bias is a loan from the architect to the optimiser. The CNN architect pre-pays the cost of discovering “edges are local” and “a cat is a cat wherever it sits” by wiring those facts into the weight-sharing pattern. The ViT architect pays nothing up front and bills the data instead: it must *observe* enough translated, localised examples to rediscover the same facts. Small dataset, large bill, overfitting.

2 Why parameter sharing matters, and why attention is permutation-equivariant

Parameter sharing. A 3×3 conv with $C_{\text{in}}=C_{\text{out}}=d$ uses $3 \cdot 3 \cdot d \cdot d = 9d^2$ weights *total*, reused at every one of the N spatial positions. A fully-connected layer over the same $N \times d$ map would use $(Nd)^2$ weights. The convolution’s reuse is exactly what enforces translation equivariance: shift the input by one pixel and, because the same kernel is applied everywhere, the output shifts by one pixel too.

Permutation equivariance of attention. Let π be any permutation of the N patch rows, applied by a permutation matrix Π . Self-attention without positional information computes $A = \text{softmax}(QK^\top / \sqrt{d_k})V$ with $Q = XW_Q$, $K = XW_K$, $V = XW_V$. Replacing $X \rightarrow \Pi X$ gives $Q \rightarrow \Pi Q$, $K \rightarrow \Pi K$, $V \rightarrow \Pi V$, so the score matrix becomes

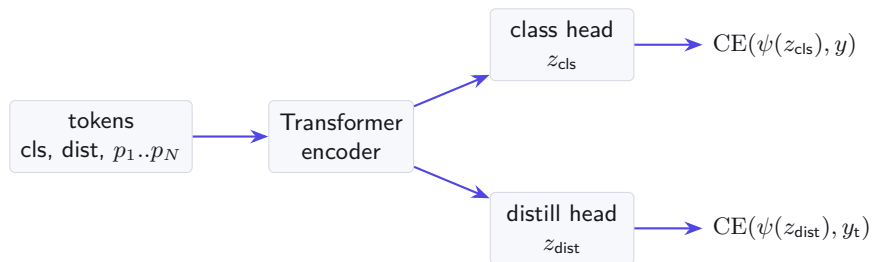
$$(\Pi Q)(\Pi K)^\top = \Pi (QK^\top) \Pi^\top,$$

the row-wise softmax commutes with the row/column permutation, and the output is ΠA — the *same* outputs, merely reordered (see Module 0.1). Attention cannot tell patch order from content alone; the positional embedding from Topic 1 is what breaks this symmetry and tells the model *where* each patch came from.

A CNN’s spatial prior is *structural* (free, fixed). A ViT’s spatial prior is *parametric* (learned, paid for in data). Permutation equivariance is the precise statement that, stripped of positional embeddings, a Transformer treats its inputs as a set — so all geometric knowledge has to be supplied, either by lots of data or by a teacher.

3 DeiT: distillation through a dedicated token

DeiT (Data-efficient image Transformer) keeps the architecture but adds a second special token — a *distillation token* x_{dist} — alongside the class token. It enters the sequence just like the class token, attends through every block, and feeds its own linear head. The class head is trained against the true label y ; the distillation head is trained against the *teacher’s* prediction y_t , where the teacher is a strong CNN.



The *hard-label* distillation objective averages the two cross-entropies, with $\psi = \text{softmax}$:

$$\mathcal{L}_{\text{hard}} = \frac{1}{2} \text{CE}(\psi(z_{\text{cls}}), y) + \frac{1}{2} \text{CE}(\psi(z_{\text{dist}}), y_t), \quad y_t = \arg \max_c f_{\text{teacher}}(x)_c \quad (2)$$

The teacher’s hard label y_t injects the CNN’s inductive bias as a training signal: the student learns to agree with a model that already knows about locality, without that bias being wired into its own architecture.

4 The derivation: a 3-class distillation loss, computed in full

Setup. Three classes $\{0, 1, 2\}$. True label $y = 0$. The student produces two logit vectors, one per head:

$$z_{\text{cls}} = (2.0, 0.5, -1.0), \quad z_{\text{dist}} = (1.0, 2.5, 0.0).$$

The CNN teacher produces logits $(0.3, 2.2, 1.1)$, so its hard label is $y_t = \arg \max(0.3, 2.2, 1.1) = 1$.

Step 1 — class-head softmax. Exponentiate (subtracting the max 2.0 for stability):

$$e^{z-2.0} = (e^0, e^{-1.5}, e^{-3.0}) = (1.0, 0.2231, 0.0498), \quad \sum = 1.2729.$$

$$\psi(z_{\text{cls}}) = \left(\frac{1.0}{1.2729}, \frac{0.2231}{1.2729}, \frac{0.0498}{1.2729} \right) = (0.7856, 0.1753, 0.0391).$$

Step 2 — class cross-entropy against $y = 0$ (picks coordinate 0):

$$\text{CE}(\psi(z_{\text{cls}}), 0) = -\ln(0.7856) = 0.2413.$$

Step 3 — distillation-head softmax. Subtract the max 2.5:

$$e^{z-2.5} = (e^{-1.5}, e^0, e^{-2.5}) = (0.2231, 1.0, 0.0821), \quad \sum = 1.3052,$$

$$\psi(z_{\text{dist}}) = (0.1710, 0.7662, 0.0629).$$

Step 4 — distillation cross-entropy against $y_t = 1$ (picks coordinate 1):

$$\text{CE}(\psi(z_{\text{dist}}), 1) = -\ln(0.7662) = 0.2664.$$

Step 5 — average.

$$\mathcal{L}_{\text{hard}} = \frac{1}{2}(0.2413) + \frac{1}{2}(0.2664) = \frac{1}{2}(0.5077) = \boxed{0.2538}.$$

4.1 The two predicted distributions as heatmaps

Each row is one head's softmax over the three classes; deeper indigo means higher probability. The class head concentrates on class 0 (the true label); the distillation head concentrates on class 1 (the teacher's pick).

	class 0	class 1	class 2
$\psi(z_{\text{cls}})$	0.786	0.175	0.039
$\psi(z_{\text{dist}})$	0.171	0.766	0.063

This is the *hard* variant: the teacher contributes only its argmax label. The *soft* variant instead matches the full teacher distribution with a temperature-scaled KL term. DeiT reports that the hard variant works at least as well and is simpler — it slots directly into the same cross-entropy machinery used for the class head.

5 Properties and consequences

1. **Two heads, one backbone.** Class and distillation tokens share all encoder weights; only their tiny linear heads differ. At inference the two head logits are typically averaged.
2. **The teacher supplies the missing bias.** A CNN teacher already encodes locality and equivariance; distilling its labels transfers that prior as supervision rather than architecture.
3. **Data efficiency.** With distillation, DeiT matches strong CNNs trained on *ImageNet-1k only* — no JFT-300M pretraining needed — which was ViT’s original requirement.
4. **Equal weighting.** The $\frac{1}{2}, \frac{1}{2}$ split treats true-label and teacher-label objectives symmetrically; in our example they happened to be close (0.2413 vs 0.2664).

Without enough data *and* without distillation, a ViT has no spatial prior to fall back on: it memorises the small training set and *underperforms* a CNN of comparable size. The fix is not “train longer” — it is to supply the bias, either through massive pretraining data or through a teacher. Distillation is cheaper than collecting 300M labelled images.

6 Where this sits in the track

Link	Relevance
Module 0.1	Permutation equivariance of attention — the symmetry argument used here.
Topic 1 (prev)	Patch embeddings and the learned positional table that break that symmetry.
Later modules	Self-supervised pretraining (e.g. masked image modelling) as another way to pay the data bill without a teacher.

A Reproducible (NumPy)

Inputs: $z_{\text{cls}}=(2.0, 0.5, -1.0)$, $z_{\text{dist}}=(1.0, 2.5, 0.0)$, $y=0$, teacher logits $(0.3, 2.2, 1.1)$ giving $y_t=1$.
 Derived: $\psi(z_{\text{cls}}) = (0.7856, 0.1753, 0.0391)$, $\psi(z_{\text{dist}}) = (0.1710, 0.7662, 0.0629)$, $\text{CE}_{\text{cls}} = 0.2413$, $\text{CE}_{\text{dist}} = 0.2664$, $\mathcal{L} = 0.2538$.

```
import numpy as np
def softmax(z):
    z = np.asarray(z, float); e = np.exp(z - z.max()); return e / e.sum()

z_cls = np.array([2.0, 0.5, -1.0])
z_dist = np.array([1.0, 2.5, 0.0])
y = 0
teacher = np.array([0.3, 2.2, 1.1])
y_t = int(teacher.argmax()) # 1

p_cls = softmax(z_cls) # [0.7856 0.1753 0.0391]
```

```
p_dist = softmax(z_dist)                # [0.1710 0.7662 0.0629]
ce_cls  = -np.log(p_cls[y])              # 0.2413
ce_dist = -np.log(p_dist[y_t])           # 0.2664
L = 0.5 * ce_cls + 0.5 * ce_dist         # 0.2538
print(np.round(p_cls,4), np.round(p_dist,4),
      round(ce_cls,4), round(ce_dist,4), round(L,4))
# -> [0.7856 0.1753 0.0391] [0.171 0.7662 0.0629] 0.2413 0.2664 0.2538
```

Marevlo Research · Transformer Track · Module 6.1 · Topic 2 of 2.